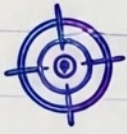


SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



1

SQL FUNDAMENTALS



★ What is SQL?

SQL = Structured Query Language.
Structured Query Language
language to communicate with database
used to store retrieve manipulate
manipulate data.

★ What is RDBMS?

Relational Database Management System
stores data in tables with rows
columns — eg MySQL PostgreSQL
Oracle SQL Server.

★ What is Database vs DBMS?

Aspect	DATABASE	DBMS
Definition:	collection of data	software manage database
Example:	My data	MySQL

★ What is Table?

Example table with green header —
Students:

Columns →

id	name	age	city
1	Aman	22	Delhi

← Rows

← Columns = vertical

← Rows = horizontal

★ What is Row vs Column?

Row = record — horizontal —

Column = field — vertical —

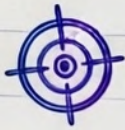
Key Terms

- Row Record — horizontal entry in table
- Column Field — vertical field in table
- Table — collection of rows in database
- Primary Key — unique identifier for each row
- RDBMS — stores data in tables (e.g. MySQL, PostgreSQL)



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



2 DATA TYPES & CONSTRAINTS



★ What are SQL Data Types?

Data Type = type of value in column.

- Numeric: - "INT, FLOAT, DECIMAL"
- String: - "VARCHAR, CHAR, TEXT"
- Date: → "DATE, TIME, DATETIME, TIMESTEMD"
- Boolean: - "BOOLEAN"

★ What are Constraints?

Rules for data

- Primary Key → yellow on "Primary Key" unique, not null, one per table
- Foreign Key → "Foreign Key"
- Unique : links two tables
- Unique unique values
- Not Null → cannot be null
- Check → condition age > 18
- Default → default value



★ Constraint vs USE

	CONSTRAINT	USE
1	Primary Key	Primary Key
2		unique identifier
2	Foreign Key	links tables
3		
4	Unique	unique values
6	Not Null	no null
7	Check	condition
8	Default	default value

Example table with green header:
Students with Constraints

id	name	age	constraints info
INT	PRIMARY KEY	NOT NULL	Delhi
1	INT CHECK	age > 0	Pune
2	Alice	20	✓ valid
3	Bob	25	✓ valid

Columns →

→ Columns = vertical fields
→ Rows = horizontal records

Rows ↓



Key Terms:

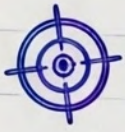


Save this guide • Share with team • Interview Tip: Practice coding on paper
Final advice: Understand → Practice • Whiteboard → Explain out loud — paper
You've got this!



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



3 SQL COMMAND TYPES - DDL DML DCL TCL



★ What is DDL?

Data Definition Language defines structure

CREATE • ALTER • DROP • TRUNCATE

Example:

```
CREATE TABLE Students (id INT PRIMARY KEY);
```

★ What is DML?

Data Manipulation Language manipulates data

SELECT • INSERT • UPDATE • DELETE

Example:

```
INSERT INTO Students VALUES (1, 'Aman', 22);
```

id	name	age
1	Aman	22
2	Priya	20

← Rows

Columns →

★ What is DCL?

Data Control Language — GRANT • REVOKE permissions.

★ What is TCL?

Transaction Control Language — COMMIT • ROLLBACK • SAVEPOINT.

COMMAND TYPE		COMMANDS	PURPOSE
DDL	CREATE	ALTER DROP	structure
DML	SELECT	INSERT UPDATE	DELETE
DCL	GRANT	REVOKE	permissions
TCL	COMMIT	ROLLBACK	transaction



Example Syntax:

- CREATE TABLE Courses (course_id INT, name VARCHAR);
- ALTER TABLE Courses ADD capacity INT;
- DROP TABLE Courses;

★ Key Terms

- DDL → structure
- DCL → control
- TCL → transaction
- DML → data



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts

SELECT - SUPER CAPABILITY

4 SECTION 4 — SELECT & FILTERING

★ What is SELECT?

- SELECT retrieves data from table
- SELECT * FROM Students selects all columns
- SELECT id, name FROM Students selects specific columns.

★ What is WHERE?

- Filters rows based on condition
- SELECT * FROM Students WHERE age > 20

WHERE = filter rows by condition

★ What is ORDER BY?

- Sorts results by column
- ORDER BY age ASC / DESC
- ASC = ascending (low → high), Example: Students table filtering → WHERE age > 20
- DESC = descending (high → low)

Example: SELECT * FROM Students
ORDER BY age DESC

	name	age	city
1	Aman	21	Delhi
2	Priya	22	Pune
3	Rohan	23	Mumbai

← Rows →

← row kept (age > 20)

★ What is DISTINCT?

Removes duplicates, returns unique values
SELECT DISTINCT city FROM Students
Only unique cities

★ What is LIMIT & OFFSET?

LIMIT 10 = first 10 rows

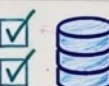
OFFSET 5 = skip 5 rows

Example: SELECT * FROM Students
LIMIT 5 OFFSET 5 → rows 6-10

CLAUSE	USE	EXAMPLE
WHERE	filter rows	WHERE age > 20
ORDER BY	sort results	ORDER BY
DISTINCT		age DESC
	unique values	DISTINCT city
LIMIT	top N rows	LIMIT 10

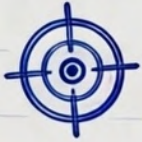
★ Key Terms

- SELECT → retrieve data
- WHERE → filter rows
- ORDER BY → sort results
- DISTINCT → unique values
- LIMIT → top N rows
- OFFSET → skip rows



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



5

SECTION NAME:

5 OPERATORS & PATTERN MATCHING



★ What are Operators?

- Comparison = **!= > < >= <=**
- Logical AND OR NOT
- Arithmetic **+ - * /**

Operators = symbols to filter / compare data in WHERE

★ What is LIKE?

Pattern matching • **%** any characters (0 or more) • **_** single character

Example:

- name LIKE **'A%'** → Starts with **A**
- name LIKE **%n'** → ends with **n**
- name LIKE **%ma%** → Contains **ma**

★ What is IN?

IN = list of values WHERE city IN ("Delhi", 'Mumbai') —
— multiple values filter

★ What is BETWEEN?

BETWEEN = range inclusive
WHERE age BETWEEN 20 AND 25

★ What is AND OR NOT?

- Combines conditions
- WHERE age > 20 AND city = 'Delhi'
- OR — either true
- NOT — negates

OPERATOR	EXAMPLE	RESULT
LIKE 'A%'	names starting with A	
IN ("Delhi", 'Mumbai')		in list
BETWEEN 20 AND 25		in range
AND		both true

Example Queries:

Example: Students

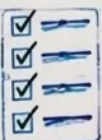
id	name	age	city
1	Aisha	22	Delhi
2	Rohan	24	Mumbai
3	Aman	21	Pune

← Rows

↓ Columns →

★ Key Terms

- LIKE = pattern • **% _** wildcards
- IN = list filter
- BETWEEN = range filter 20 AND 25
- AND OR NOT = combine conditions
- Operators = comparison / logical / arithmetic



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts

Page 6/20



6 AGGREGATE & GROUP BY



★ What are Aggregate Functions?

Aggregate = performs calculation on set of values → returns single value.

- COUNT counts rows COUNT(*) all
COUNT(col) non-null
- SUM sum values SUM(age)
- AVG average AVG(age)
- MIN minimum
- MAX maximum

★ What is GROUP BY?

Groups rows by column → SELECT city, COUNT(*)
FROM Students GROUP BY city,
counts per city.

★ What is HAVING?

- Filters groups after GROUP BY.
 - WHERE filters rows before grouping
 - HAVING filters groups after grouping
- HAVING COUNT(*) > 2

FUNCTION	USE	EXAMPLE
COUNT	counts	COUNT(*)
SUM	sum	SUM(salary)
AVG	average	AVG(age)
MIN	minimum	MIN(age)

WHERE vs HAVING	
WHERE filters rows before grouping	HAVING filters after grouping
Cannot use aggregate	Can use aggregate

Example —

```
SELECT city, AVG(age)
FROM Students GROUP BY city
HAVING AVG(age) > 22.
```

★ Key Terms

- COUNT SUM AVG MIN MAX,
- GROUP BY groups,
- HAVING filters groups

SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



7 JOINS PART 1



★ What is JOIN?

Combines rows from two tables based on related column.

★ What is INNER JOIN?

Returns matching rows in both tables.

```
SELECT * FROM Students
INNER JOIN Courses
ON Students.course_id=Courses.id
only matching
```

★ What is LEFT JOIN?

Returns all left table + matching right, null if no match.

```
SELECT * FROM Students
LEFT JOIN Courses
all students even if no course.
```

Example tables:

Students		
id	name	course_id
1	Alice	101
2	Bob	102
3	Cara	103

↓
Courses

JOIN on course_id →

★ What is RIGHT JOIN?

Returns all right table + matching left.

★ What is FULL JOIN?

Returns all from both tables — matching + non-matching.

JOIN TYPE	RESULT	EXAMPLE
INNER	matching both	only matches
LEFT	all left + matching right	all students...
RIGHT	all right + matching left	all courses...
FULL	all both	every row

Venn Diagram:



INNER



LEFT



RIGHT



FULL



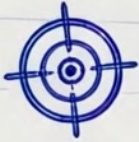
Tip: JOIN uses a common column to connect tables, e.g. course_id

★ Key Terms

- JOIN combines tables
- INNER = matching
- LEFT = all left
- RIGHT = all right
- FULL = all

SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



8

SECTION NAME: 8 JOINS PART 2 & UNION



★ Q: What is CROSS JOIN?

definition:

- CROSS JOIN = 'Cartesian product' → all combinations.

- SELECT * FROM Students CROSS JOIN Courses.

If 3 students, 2 courses = 6 rows.

→ every row in table1 is paired with every row in table2

★ Q: What is SELF JOIN?

Table joins itself.

- SELECT A.name, B.name FROM Employees A JOIN Employees B ON A.manager_id = B.id.
- employee and manager are in same table.

A is the employee, B is their manager — both from Employees table via manager_id

★ Q: What is difference JOIN vs UNION

Aspect	JOIN	UNION
Combines	columns horizontally	rows vertically
Output	adds columns to table	adds rows to
Column count	may differ columns	number

★ Q: What is UNION vs UNION ALL?

Definition:

- UNION removes duplicates, UNION ALL keeps duplicates, UNION ALL faster.

UNION vs UNION ALL:		Performance
UNION	removes duplicates	slower
UNION ALL	keeps duplicates	faster



Example - Code:

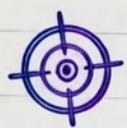
- SELECT id FROM Students
- SELECT id FROM Students
- UNION ALL SELECT id FROM Teachers; -- keeps duplicates

★ Key Terms

- CROSS → cartesian → same table
- JOIN → vertical
- SELF → horizontal → UNION ALL → faster

SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



9



SECTION



SUBQUERIES



★ What is Subquery?

Subquery = nested query. A query inside another query.
 SELECT * FROM Students WHERE age > (SELECT AVG(age) FROM Students) outer query uses inner result.

Types:

Non-correlated - independent of outer

Correlated - depends on outer (yellow highlight depends, nested query)

★ What is Nested Subquery?

Example SELECT name FROM Students WHERE course_id IN (SELECT id FROM Courses WHERE fee > 5000).



Nested inside FROM or WHERE clause
 - Can return single value or multiple values

★ What is Correlated Subquery?

Inner references outer.

SELECT name FROM Students S
 WHERE EXISTS (SELECT 1 FROM Courses C WHERE C.id = S.course_id AND C.fee > 5000)
 runs for each outer row.

NON-CORRELATED	CORRELATED
Independent	Depends on outer
Executes once	Executes for each row
Faster	Slower

★ What is EXISTS vs IN?

EXISTS checks existence faster for large data, IN checks list.



Use EXISTS for large tables
 → performance

★ Key Terms

- Subquery = query inside query
- Non-correlated = independent
- Correlated = depends outer
- EXISTS = existence check



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



10

WINDOW FUNCTIONS



★ What is Window Function?

Calculates value across a set of rows related to the current row.

Keywords:

- OVER() clause
- OVER(PARTITION BY department)
- Window = OVER(PARTITION BY ... ORDER BY ...)

★ What is PARTITION BY?

Divides result set into partitions/groups for the window.

- ROW_NUMBER: unique rank 1,2,3
- RANK: same rank with gaps 1,1,3
- DENSE_RANK: same rank no gaps 1,1,2
- LEAD: next row
- LAG: previous row

FUNCTION	USE	EXAMPLE
ROW_NUMBER	unique rank	ROW_NUMBER()
RANK	same rank with gaps	RANK()
DENSE_RANK	same rank no gaps	DENSE_RANK()
LEAD / LAG	next / previous	LEAG(salary)

Example Table: employees

id	name	age	city	salary	department
1	Alice	25	NYC	7000	Sales
2	Bob	30	SF	8000	Engineering
3	Charlie	28	NYC	6000	Sales
4	Diana	35	LA	9000	HR

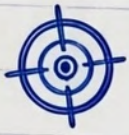
★ Key Terms

- OVER() defines window
- PARTITION BY groups
- ROW_NUMBER unique
- RANK gaps
- DENSE_RANK no gaps



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



11

11 CTE & TEMP TABLES



★ What is CTE?



CTE = Common Table Expression → temporary named result defined with WITH clause. Makes query readable & reusable.

```
WITH AvgAge AS (SELECT AVG(age) AS avg_age
FROM Students)
SELECT * FROM Students
WHERE age > (SELECT avg_age FROM AvgAge)
```

★ What is difference CTE vs Subquery?

CTE → defined with WITH, reusable + readable. Can be referenced multiple times in main query.

Subquery → inline inside SELECT/WHERE, not reusable, runs once per outer query.



★ What is Temp Table?

Temp Table = CREATE TEMPORARY TABLE TempStudents stores intermediate results, session specific.

★ What is difference CTE vs Temp Table?

CTE → not stored on disk, temporary in memory, disappears after query execution.

Temp Table → stored on disk, persists for session, can be indexed, reused multiple times.

CTE vs SUBQUERY vs TEMP TABLE

Aspect	CTE	Subquery	Temp Table
Syntax	WITH ... AS (...)	Nested inside query	CREATE TEMPORARY TABLE ...
Reusable	reusable readable	inline not reusable	reusable in session
Storage	temporary memory	temporary memory	stored on disk session

★ Example: Using CTE for filtered query:

```
WITH HighFee AS (SELECT * FROM Courses
WHERE fee > 5000)
SELECT * FROM Students
WHERE course_id IN (SELECT id FROM HighFee)
```

Key Terms



- CTE → WITH clause, reusable
- Subquery → inline, not reusable
- Temp Table → session stored, persist until session ends
- WITH clause → makes query readable & maintainable



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



12

INDEXES



★ What is Index?

Index = Data structure speeds up retrieval like book index avoids full table scan.

Types:

- Clustered: one per table — sorts table
- Primary Key creates clustered
- Non-clustered: separate structure points to data → many per table

★ How Index works?

B-Tree balanced tree $O(\log n)$ search.
Root → Branches → Leaves.
Fast lookup!

★ When to create Index?

- On columns used in WHERE JOIN ORDER BY frequently searched.

★ When NOT to create?

CLUSTERED vs NON-CLUSTERED

- On small tables • frequent INSERT UPDATE DELETE → overhead.

★ What is Composite Index?

Index on multiple columns → CREATE INDEX idx_name_city ON Students(name, city)
Order matters! Good for queries filtering both.

```
CREATE INDEX idx_name_city
ON Students(name, city)
```

★ What is Full-Text Index?

For text search on VARCHAR/TEXT.
Uses FULLTEXT keyword. Example:
searching words in description.
LIKE %word% slow, Full-Text fast.



Example: Students table —
columns: id, name, city.
Use for text search queries.



Aspect	Clustered	Non-Clustered
One per table	One per table	Many per table
Sorts table	Separate structure	CREATE INDEX creates



★ Key Terms:

- Index speeds retrieval, avoids scan
- Clustered — sorts table, one per table
- Non-clustered — separate, many per table, points to data
- B-Tree structure, $O(\log n)$ search
- Composite = multi-column
- Full-Text = for text search

SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



13 VIEWS & PROCEDURES



★ View = Virtual table based on SELECT query

- Does NOT store data; stores the query only

- Example: `CREATE VIEW AdultStudents AS SELECT * FROM Students WHERE age >= 18`

★ Materialized View = Stores data physically

- Data is stored physically on disk; requires refresh

- Faster for complex queries; use `REFRESH` to update

★ Stored Procedure = Precompiled SQL code reusable

- Example: `CREATE PROCEDURE GetStudents @age INT AS`

`SELECT * FROM Students WHERE age > @age`

- Call with: `EXEC GetStudents 20`

★ Function vs Procedure

- Function: Returns a value; can be used in `SELECT`

- Procedure: May NOT return; can have `OUT` params

VIEW	MATERIALIZED VIEW	TABLE
No storage	Stores data	Stores data
Virtual / no data	Stores data	Stores data
Auto updated	Manual refresh	Direct update

★ FUNCTION vs PROCEDURE

- Returns value vs May not return

- Used in `SELECT` vs `EXEC`

- Must return vs May return via `OUT`

★ Key Terms ★

- View: virtual table

- Materialized: stores data

- Procedure: precompiled

- Function: returns value

SQL INTERVIEW HANDBOOK

50 Must-Know Concepts

Page 14/20

14



SECTION NAME: 14 NORMALIZATION



★ What is Normalization?

Organizes data to reduce redundancy
improve integrity.

★ What is 1NF?

1NF — First Normal Form

- atomic values — no repeating groups
each cell holds single value
- Example: Students with multiple hobbies
stored as hobby1, hobby2 → not 1NF
- ✓ fix: split into separate table
(Student_ID, Hobby)

★ What is 2NF?

2NF — Second Normal Form

- 1NF + no partial dependency
- all non-key attributes depend on the
whole primary key
- Example: Students_Courses with composite key
(student_id, course_id) → remove partial
- ✓ Goal: ensure each non-key depends on full key

★ What is 3NF?

3NF — Third Normal Form

- 2NF + no transitive dependency
- non-key attribute does not depend on
another non-key

★ What is BCNF?

BCNF — Boyce-Codd Normal Form

- stronger than 3NF: every determinant is a
candidate key

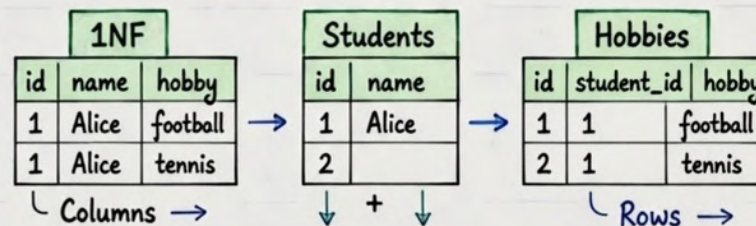
★ What is Denormalization?

- Adding redundancy for performance
(read-heavy systems)



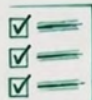
NORMAL FORM	RULE	PURPOSE
1NF	atomic, no repeating groups eliminate repeating groups	
2NF	1NF + no partial dependency	eliminate partial
3NF	2NF + no transitive dependency	
3NF	every determinant = candidate key	
BCNF	ensure clean dependencies	

Example: Students progression → to 3NF / split →



Key Terms

- Normalization: reduce redundancy
- 1NF: atomic
- 2NF: no partial
- 3NF: no transitive
- BCNF: every determinant is candidate key
- Denormalization: for performance



SQL INTERVIEW HANDBOOK

50 MUST-KNOW CONCEPTS

15 TRANSACTIONS & ACID

Page 15/2



★ What is Transaction?

- Definition: Transaction = Single unit of work, all or nothing
- BEGIN COMMIT ROLLBACK

Ensures all operations succeed as a single unit, or none are applied.

Example:

- BEGIN
- INSERT INTO orders (user_id, amount) VALUES (101, 100);
- UPDATE balance SET amount = amount - 50 WHERE user_id = 101;
- COMMIT;
- if error → ROLLBACK;

★ What is Lock?

Lock mechanism prevents concurrent modification.

- Shared lock (read): allows multiple reads, blocks writes
- Exclusive lock (write): allows one write, blocks others

★ Key Terms

- Transaction = unit of work
- ACID = reliability properties
- Isolation Levels = control concurrency
- Lock = shared (read) / exclusive (write)

★ What is ACID?

ACID = set of properties for reliable transaction

ACID = Atomicity • Consistency • Isolation • Durability

- Atomicity: all or nothing — entire transaction succeeds or fails
- Consistency: database moves from valid state to valid state
- Isolation: concurrent transactions are independent
- Durability: committed data persists even if system crashes.

★ What is Isolation Levels?

Isolation levels define how transactions interact

- Read Uncommitted: dirty reads allowed — lowest isolation
- Read Committed: no dirty reads — prevents dirty reads
- Repeatable Read: no dirty, no non-repeatable reads
- Serializable: strictest — no dirty, no non-repeatable reads, no phantom

ISOLATION LEVEL	DIRTY	NON-REPEATABLE	PHANTOM
Read Uncommitted	allows all	allows all	allows all
Read Committed	no dirty	allows others	allows all
Repeatable Read	no dirty	no non-repeatable	allows phantom
Serializable	none	none	none

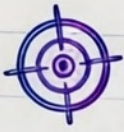


Tip: Serializable is the safest — it prevents all issues but is slowest



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



16 KEYS & RELATIONSHIPS



★ What is Primary Key?

Unique + not null, one per table, identifies row, id
PRIMARY KEY.

★ What is Super Key?

Superset of candidate key.
id+name = super key. All super keys include candidate key.

★ What is Composite Key?

Combination of columns as primary key
- (student_id, course_id) together unique. Example junction table.

★ What is Foreign Key?

Links two tables, REFERENCES other table's primary key, Students.course_id
REFERENCES
Courses.id

★ What is Relationship?

- One-to-One: one student one passport
- One-to-Many: one course many students
- Many-to-Many: many students many courses + junction table
StudentCourses

★ What is Candidate Key?

All possible unique keys - id, email both candidate. Minimal unique.

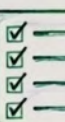
KEY TYPE vs DEFINITION vs EXAMPLE

KEY TYPE	DEFINITION	EXAMPLE
Primary Key	unique not null	id
Foreign Key	links tables	course_id
Candidate Key	possible unique	email
Composite Key	combo	(student_id, course_id)

ER Diagram example: Students || --o{ Courses (one to many) →
→ one student → many courses; one course can have many students

★ Key Terms:

- Primary → unique, not null, one per table
- Foreign → reference, links tables
- Candidate → possible unique
- Composite → combo of columns
- Relationship → 1-1, 1-M, M-M
- Super Key → superset of candidate



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts

17 ADVANCED SQL FUNCTIONS

★ What is **UNION** vs **UNION ALL**?

- **UNION** removes duplicates
- **UNION ALL** keeps duplicates — faster.

★ What is **CASE**?

- Conditional logic
- **CASE WHEN** age >= 18 **THEN** Adult **ELSE** Minor **END**

★ What is **COELEECE**?

- Must be spelled **C-O-A-L-E-S-C-E**.
- Returns first non-null.

★ What is **NULL** handling?

- **NULL** is unknown — not equal to anything — use **IS NULL** check.

★ What is **NULL** vs Empty?

- **NULL** = unknown vs empty string = "" (difference).

FUNCTION	USE	EXAMPLE
UNION	Combine results, remove duplicates	SELECT name, city FROM A UNION SELECT name, city FROM B;
UNION ALL	Combine results, keep duplicates (faster)	SELECT name, CASE WHEN age >= 18 THEN "Adult" ELSE "Minor" FROM Users;
CASE	Conditional logic	SELECT name, COOLESCCE (phone, 'No phone') FROM Students; END
COALESCE	Returns first non-null value	SELECT * FROM Orders WHERE refund_date IS NULL;
IS NULL	Check for NULL values	

★ Key Terms:

- **UNION** — combine sets, **UNION** removes duplicates
- **CASE** — conditional logic: **CASE WHEN ... THEN ... ELSE ... END**
- **COALESCE** — **C O A L E S C E** = returns first non-null
- **NULL** — unknown value, not equal to anything
- **IS NULL** — check for NULL values

SQL INTERVIEW HANDBOOK

50 Must-Know Concepts

18 PERFORMANCE & OPTIMIZATION

★ What is EXPLAIN?

Shows query execution plan
EXPLAIN SELECT * FROM Students
WHERE age > 20 → shows
 index usage / full scan

★ What is Query Optimization?

Techniques to make query faster.

1. Use **Index** on **WHERE**, **JOIN**, **ORDER BY** columns
2. Avoid **SELECT *** → select only needed columns
3. Use **LIMIT**
4. Avoid functions on indexed columns →
WHERE YEAR(date) slow vs date range fast
5. Use **JOIN** instead of subquery where possible
6. Use **UNION ALL** instead of **UNION** if duplicates ok

★ What is N+1 Problem?

One query for list + N queries for each item → causes performance issue
 Fix with **JOIN** or **eager loading**

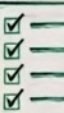
OPTIMIZATION	BAD	GOOD
SELECT *	SELECT id, name	SELECT id, name
No index	WHERE age > 20 no index	WHERE age > 20 with index
Function on column	YEAR(date) = 2024	date BETWEEN 2024-01-01 AND 2024-12-31

Example - EXPLAIN output

Code	Output					
EXPLAIN SELECT id, name FROM Students WHERE age > 20;						
id	select_type	table	type	possible_keys	rows	Extra
1	SIMPLE	Students	ALL	NULL	1000	10000
1	SIMPLE	Students	RANGE	idx_age	50	
1	SIMPLE	Students	AL	Using index condition		

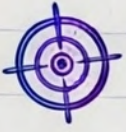
★ Key Terms:

- **EXPLAIN** → execution plan
- **Index** → speeds lookup, reduces scan time
- **SELECT *** → avoid, only fetch needed columns
- **N+1** → one + N queries problem, use **JOIN/eager load**
- **JOIN vs Subquery** → **JOIN** usually faster, fewer scans



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



19 INTERVIEW TRAPS - MUST KNOW

★ What is DELETE vs TRUNCATE vs DROP?

DELETE vs TRUNCATE vs DROP

Aspect	DELETE	TRUNCATE	DROP
Definition	removes rows WHERE allowed	removes all rows	removes table structure
Command	DML	DDL	DDL
Rollback	can rollback	cannot rollback in some DB	cannot rollback in some
Speed	slower	faster	fastest
WHERE condition	WHERE allowed	cannot use WHERE	N/A



WARNING: DELETE vs TRUNCATE vs DROP is favorite interview question.

★ What is WHERE vs HAVING?

WHERE filters rows before GROUP BY — cannot use aggregate

e.g. WHERE age → 20

HAVING filters groups after GROUP BY — can use aggregate

e.g. HAVING COUNT(*) → 2

★ What is COUNT(*) vs COUNT(col)?

COUNT(*) counts all rows including null

COUNT(col) counts non-null of that column

COUNT(*) — faster

★ What is IN vs EXISTS?

IN checks list — small data

EXISTS checks existence — large data

EXISTS faster

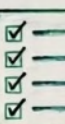


WHERE vs HAVING

Feature	WHERE	HAVING
Purpose	filters rows before grouping	filters groups after grouping
Aggregate usage	no aggregate	can aggregate
Applied to	individual rows	groups of rows

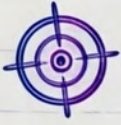
★ Key Terms

- DELETE → DML, WHERE allowed, rollback
- TRUNCATE → DDL, no WHERE, faster
- DROP → removes table
- WHERE vs HAVING → row vs group
- COUNT(*) vs COUNT(col) → all rows vs non-null
- IN vs EXISTS → small vs large



SQL INTERVIEW HANDBOOK

50 Must-Know Concepts



20 FINAL RECAP - SQL INTERVIEW



★ Fundamentals & Data Types

- SQL: language for RDBMS
- Table | Row | Column | etc

★ Operators & Clauses

- SELECT WHERE ORDER BY DISTINCT LIMIT

★ Joins & Unions

- INNER | LEFT | RIGHT | FULL | CROSS
| SELF | UNION | UNION ALL

Full 50 Concepts Checklist ✓

- ✓ Data Types | Constraints | Primary | Foreign | Unique | Not Null | Check | Default
- ✓ Command Types: DDL | DML | DCL | TCL
- ✓ Aggregate: COUNT | SUM | AVG | MIN | MAX | GROUP BY | HAVING
- ✓ Subqueries: Non-correlated | Correlated | EXISTS | IN
- ✓ Window Functions: ROW_NUMBER | RANK | DENSE_RANK | LEAD | LAG | PARTITION BY
- ✓ CTE (WITH) | Temp Table | Indexes: Clustered | Non-clustered
- ✓ Views | Materialized Views | Procedures | Functions
- ✓ Normalization: 1NF | 2NF | 3NF | BCNF | Denormalization
- ✓ Transactions: ACID | Isolation Levels | Locks
- ✓ Keys: Candidate | Super | Composite | 1-1 | 1-M | M-M
- ✓ Advanced: CASE | COLECE | NULL | UNION | Performance:
- ✓ Traps: DELETE | TRUNCATE | DROP | WHERE | HAVING | COUNT

★ Top 15 Interview Questions

No.	Question
1	Difference DELETE vs TRUNCATE vs DROP?
2	INNER vs LEFT JOIN?
3	WHERE vs HAVING?
4	What is Index?
5	ACID properties?
6	Primary vs Foreign Key?
7	UNION vs UNION ALL?
8	What is Window Function?
9	CTE vs Subquery?
10	How to optimize slow query?

★ Topic vs Status — Completed

Topic	Status
Joins	✓ Completed
Indexes	✓ Completed
Window Functions	✓ Completed
CTE & Subqueries	✓ Completed
Normalization / ACID	✓ Completed
Transactions & Locks	✓ Completed

Interview Tip:

- Save this guide | Share with team | Interview Tip: Practice queries on paper
- Final advice: Understand → Practice → Whiteboard → Explain out loud.

You've got this! ✨

